



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 04: : : Write a program to match whether a given string is generated by a Given pattern or Not

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Performance Date	11.03.2026
Submission Date	28.03.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment No:04

Experiment Name

Write a program to match whether a given string is generated by a given pattern or Not.

Objective

This experiment's goal is to create and execute a program that determines whether a given input string matches a particular regular expression pattern. Understanding pattern matching, regular expressions, and their use in lexical analysis and compiler design is aided by this experiment.

Algorithm

1. Start the program.
2. Read the number of test cases.
3. For each test case:
 - (a) Read the regular expression pattern (R).
 - (b) Read the input string (S).
 - (c) Create a regular expression object using the given pattern.
 - (d) Match the input string against the regular expression.
 - (e) If the string fully matches the pattern, display "Match Found".
 - (f) Otherwise, display "No Match".
4. Stop the program.

Source Code

```
1  #include <iostream>
2  #include <regex>
3  using namespace std;
4
5  int main() {
6      int t;
7      string pattern, str;
8
9      // Read number of test cases
10     cout << "Enter number of test cases: ";
11     cin >> t;
12
13     while (t--) {
14         // Read regex pattern
15         cout << "Enter regular expression: ";
16         cin >> pattern;
17
18         // Read input string
19         cout << "Enter string to match: ";
20         cin >> str;
21
22         // Create regex object
23         regex r(pattern);
24
25         // Match string with regex
26         if (regex_match(str, r)) {
27             cout << "Match Found" << endl;
28         } else {
29             cout << "No Match" << endl;
30         }
31     }
32
33     return 0;
34 }
35
```

Figure 1: Source code for string pattern matching using regular expressions

Output

```
1  SAKIB Code
2  cd "c:\PU Projects\PUC Courses\8th Semester\CC Lab\Lab 04\Code\" ; if ($?) {
  g++ final.cpp -o final } ; if ($?) { .\final }
3  Enter number of test cases: 1
4  Enter regular expression: a*bb
5  Enter string to match: aaabb
6  Match Found
```

Figure 2: Output showing whether the string matches the given pattern

Discussion

This task shows how to use regular expressions to determine whether a given string is produced by a particular pattern. Using built-in regex libraries, the application correctly verifies whole string matches. This method simplifies the recognition of lexical patterns and offers guidance on the implementation of regular expressions in text processing and compiler design.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 05: : : Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment Name

Write a program to identify unique identifiers from a C program using a basic lexical analyzer

Objective

This experiment aims to create and build a simple lexical analyser that finds valid identifiers by scanning a C program. While ignoring keywords, incorrect tokens, and duplication, the algorithm finds unique identifiers. This aids in comprehending how lexical analysis functions in compiler design.

Algorithm

1. Start the program.
2. Read the input C program from a file or user input.
3. Define a list of C keywords (such as int, float, char, if, while, return, etc.).
4. Initialize an empty set or list to store unique identifiers.
5. Read the program line by line.
6. For each line, break the line into tokens using delimiters such as spaces, commas, semicolons, parentheses, and operators.
7. For each token:
 - (a) Check if the token starts with a letter (a–z or A–Z) or an underscore ().
 - (b) Check if the remaining characters of the token contain only letters, digits, or underscores.
 - (c) If the token does not satisfy identifier rules, skip it.
 - (d) Compare the token with the list of C keywords.
 - (e) If the token matches a keyword, ignore it.
 - (f) Otherwise, check whether the identifier is already stored.
 - (g) If it is not present, add it to the list of unique identifiers.
8. Continue scanning until the entire program is processed.
9. Display all unique identifiers found in the C program.
10. Stop the program.

Source Code

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  bool isKeyword(const string &word)
4  {
5      static const vector<string> keywords = {
6
7          "auto", "break", "case", "char", "const", "continue", "default", "do", "double",
8          "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
9          "return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef",
10         "union", "unsigned", "void", "volatile", "while"
11     };
12
13     for (const auto &k : keywords)
14         if (word == k)
15             return true;
16
17     return false;
18 }
19 bool isIgnored(const string &word)
20 {
21     static const vector<string> ignored = {
22         "include", "stdio", "iostream", "printf", "scanf",
23         "cin", "cout", "endl"
24     };
25
26     for (const auto &w : ignored)
27         if (word == w)
28             return true;
29
30     return false;
31 }
32 bool isValidIdentifier(const string &word)
33 {
34     if (word.empty() || !(isalpha(word[0]) || word[0] == '_'))
35         return false;
36
37     for (char c : word)
38         if (!(isalnum(c) || c == '_'))
39             return false;
40
41     return true;
42 }
43 int main()
44 {
45     ifstream file("input.c");
46     map<string, int> frequency;
47     string word;
48
49     while (file >> word)
50     {
51         word = cleanToken(word);
52
53         if (isValidIdentifier(word) &&
54             !isKeyword(word) &&
55             !isIgnored(word))
56         {
57             frequency[word]++;
58         }
59     }
60     for (const auto &entry : frequency)
61     {
62         cout << left << setw(20)
63             << entry.first
64             << entry.second << "\n";
65     }
66
67     return 0;
68 }
69

```

Figure 1: Source code for lexical analyzer to detect unique identifiers

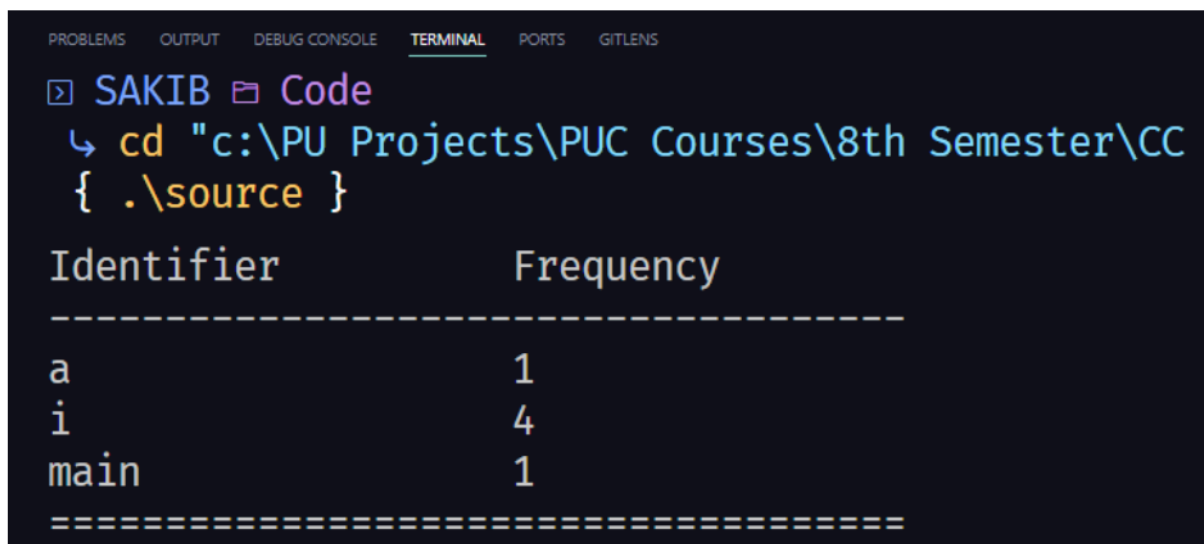
```

1  #include <stdio.h>
2
3  int main()
4  {
5      int i;
6      int a[10];
7      for(i = 1; i <= 10; i++)
8      {
9          printf("Bangladesh\n");
10     }
11
12     return 0;
13 }

```

Figure 2: Input code for lexical analyzer to detect unique identifiers

Output



Identifier	Frequency
a	1
i	4
main	1

Figure 3: Output showing all unique identifiers extracted from the C program

Discussion

The design of a basic lexical analyser to extract identifiers from a C program is demonstrated in this project. After scanning the input, the analyser applies identification rules, eliminates keywords, and uses an appropriate data structure to guarantee uniqueness. Tokenisation, identifier recognition, and the significance of lexical analysis in the initial stage of a compiler are all clearly explained by this experiment.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 06: : : Write a program to detect and remove left recursion from a given context-free grammar using C.

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment Name

Write a program to detect and remove left recursion from a given context-free grammar using C.

Objective

This experiment's goal is to create and construct a C program that takes a context-free grammar as input, looks for left recursion in the products, and eliminates it by changing the grammar into a non-left-recursive counterpart. This aids in comprehending the significance of removing left recursion prior to using top-down parsing strategies in compiler design.

Algorithm

1. Start the program.
2. Read the number of grammar productions or read rules until an empty line.
3. For each production of the form $A \rightarrow \alpha$:
 - (a) Extract the left-hand side (non-terminal) and right-hand side.
 - (b) Split the right-hand side into individual productions using '—' as delimiter.
 - (c) Separate the productions into two groups:
 - Those starting with the same non-terminal A (left recursive - α part).
 - Those not starting with A (β part).
 - (d) If left recursive productions exist:
 - Print the original production and indicate left recursion is found.
 - Transform the grammar using the standard left recursion removal rule:
$$\begin{array}{l} A \rightarrow \beta A' \quad | \beta \\ A' \rightarrow \alpha A \quad | \epsilon \end{array}$$
 - (e) If no left recursion is found, print the production as it is.
4. Continue until all productions are processed.
5. Stop the program.

Source Code

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5  #define LEN 100
6
7  void removeLR(char g[MAX][LEN], int n) {
8      for (int i = 0; i < n; i++) {
9          char L[LEN], R[LEN], *p, prod[10][LEN], a[10][LEN], b[10][LEN];
10         int pc = 0, ac = 0, bc = 0;
11
12         sscanf(g[i], "%[^-]->%[^\\n]", L, R);
13
14         p = strtok(R, "|");
15         while (p) { strcpy(prod[pc++], p); p = strtok(NULL, "|"); }
16
17         for (int j = 0; j < pc; j++)
18             if (!strcmp(prod[j], L, strlen(L)))
19                 strcpy(a[ac++], prod[j] + strlen(L));
20             else
21                 strcpy(b[bc++], prod[j]);
22
23         if (ac) {
24             printf("\nLeft Recursion in %s\n", L);
25             for (int j = 0; j < bc; j++)
26                 printf("%s%s", b[j], L, j < bc - 1 ? " | " : "");
27
28             printf("\n%s' -> ", L);
29             for (int j = 0; j < ac; j++)
30                 printf("%s%s", a[j], L, j < ac - 1 ? " | " : "");
31
32             printf(" | ε\n");
33         } else {
34             printf("\nNo left recursion: %s\n", g[i]);
35         }
36     }
37 }
38
39 int main() {
40     char g[MAX][LEN];
41     int n = 0;
42
43     printf("Enter grammar (empty line to stop):\n");
44     while (fgets(g[n], LEN, stdin) && g[n][0] != '\n') {
45         g[n][strlen(g[n], "\n")] = 0;
46         n++;
47     }
48
49     removeLR(g, n);
50     return 0;
51 }

```

Figure 1: Source code for detecting and removing left recursion from a given grammar

Output

```
Enter grammar rules (press ENTER on empty line to finish):  
S->S0S1S|01
```

```
Left Recursion Found in S
```

```
Grammar after removing left recursion:
```

```
S -> 01S'
```

```
S' -> 0S1SS' | 01
```

```
Process returned 0 (0x0)   execution time : 103.087 s
```

```
Press any key to continue.
```

```
|
```

Figure 2: Output showing detection and removal of left recursion

Discussion

The implementation of left recursion removal, a critical step in preparing a language for predictive (top-down) parsing, is demonstrated in this experiment. By determining if any alternative on the right side starts with the same non-terminal as the left-hand side, the software accurately detects left-recursive productions. The language produced by the grammar is then preserved but direct left recursion is eliminated by applying the standard transformation. The implementation is effective for laboratory use because string tokenisation using `strtok()` and careful parsing of productions are used. This exercise gives participants a practical grasp of the grammar transformation methods employed in a compiler's syntax analysis stage.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Experiment 07: : : Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Submitted by:

Name	Sayed Hossain
ID	0222210005101102
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chittagong

Remarks

Experiment Name:

Write a program to apply Algorithm for Left Factoring a Context-Free Grammar (CFG).

Objective

Implementing a software that does left factoring on a given Context-Free Grammar (CFG) is the aim of this experiment. One grammar modification method for getting rid of common prefixes is left factoring from productions. By eliminating uncertainty in production options, this transformation helps make the grammar appropriate for top-down parsing techniques like recursive descent parsing.

Algorithm

Step 1: Include required headers and declare variables for grammar, LHS, and RHS productions.

Step 2: Take grammar input (e.g., $A \rightarrow abC|abD|aE$).

Step 3: Extract the left-hand side (non-terminal).

Step 4: Remove LHS and arrow to get the right-hand side.

Step 5: Split RHS using | into individual productions.

Step 6: Store productions in a list/vector.

Step 7: Check for common prefix among productions.

Step 8: If found, separate prefix (α) and suffixes (β).

Step 9: Apply left factoring:

- $A \rightarrow \alpha A'$
- $A' \rightarrow \beta_1 | \beta_2 | \dots | \beta$

Step 10: Replace empty suffix with ϵ .

Step 11: Display the new grammar.

Step 12: If no prefix exists, show “No left factoring needed.”

Step 13: End program.

Source Code:

```
#include <iostream>
#include <vector>
using namespace std;

string prefix(string a, string b){
    string p="";
    for(int i=0; i<min(a.size(), b.size()) && a[i]==b[i]; i++)
        p+=a[i];
    return p;
}

int main(){
    string g, t="";
    cout<<"Enter production (like: A->abC|abD|aE): ";
    cin>>g;

    char A=g[0];
    vector<string> r;

    for(int i=3; i<g.size(); i++){
        if(g[i]=='|'){ r.push_back(t); t=""; }
        else t+=g[i];
    }
    r.push_back(t);

    string p=r[0];
    for(int i=1; i<r.size(); i++)
        p=prefix(p, r[i]);

    if(p==""){ cout<<"No Left Factoring needed\n"; return 0; }

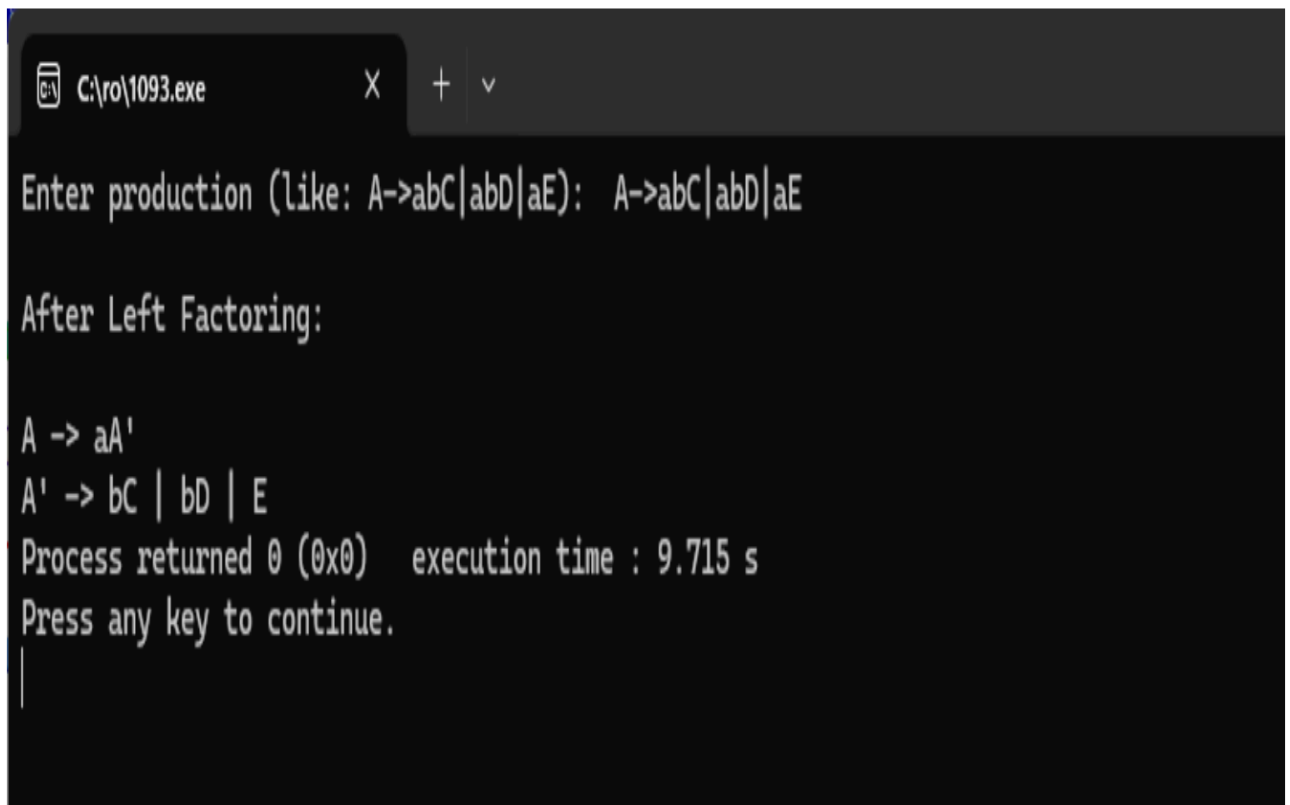
    cout<<"\nAfter Left Factoring:\n\n";
    cout<<A<<" -> " << p << A << "'";

    for(string s:r)
        if(s.substr(0, p.size())!=p)
            cout<<" | " << s;

    cout<<"\n" << A << "' -> ";

    for(int i=0; i<r.size(); i++){
        string suf=r[i].substr(p.size());
        cout<<(suf=="?"?"ε":suf);
        if(i!=r.size()-1) cout<<" | ";
    }
}
```


Output:



```
C:\ro\1093.exe X + v
Enter production (like: A->abC|abD|aE): A->abC|abD|aE

After Left Factoring:

A -> aA'
A' -> bC | bD | E
Process returned 0 (0x0) execution time : 9.715 s
Press any key to continue.
|
```

Discussion

The left factoring technique used in compiler design is demonstrated in this experiment. Because it eliminates ambiguity in grammatical productions that share a prefix, left factoring is crucial. Predictive parsers find it challenging to decide which production rule to apply during parsing because of these ambiguities. The grammar is improved for top-down parsing methods like recursive descent parsing by factoring away the common prefix and adding a new non-terminal symbol. Students learn about grammar transformation and the function of left factoring in syntax analysis through this experiment.